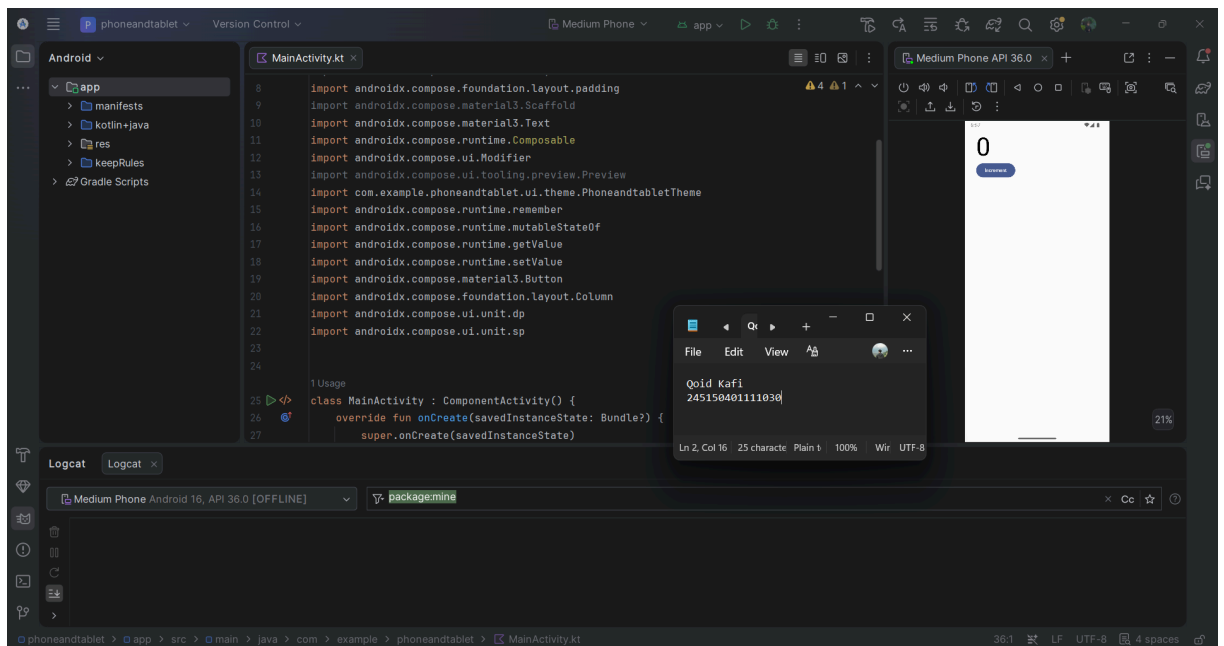


NAMA : QOID KAFI
NIM : 245150401111030
KELAS : SI - C
BAB : IV –

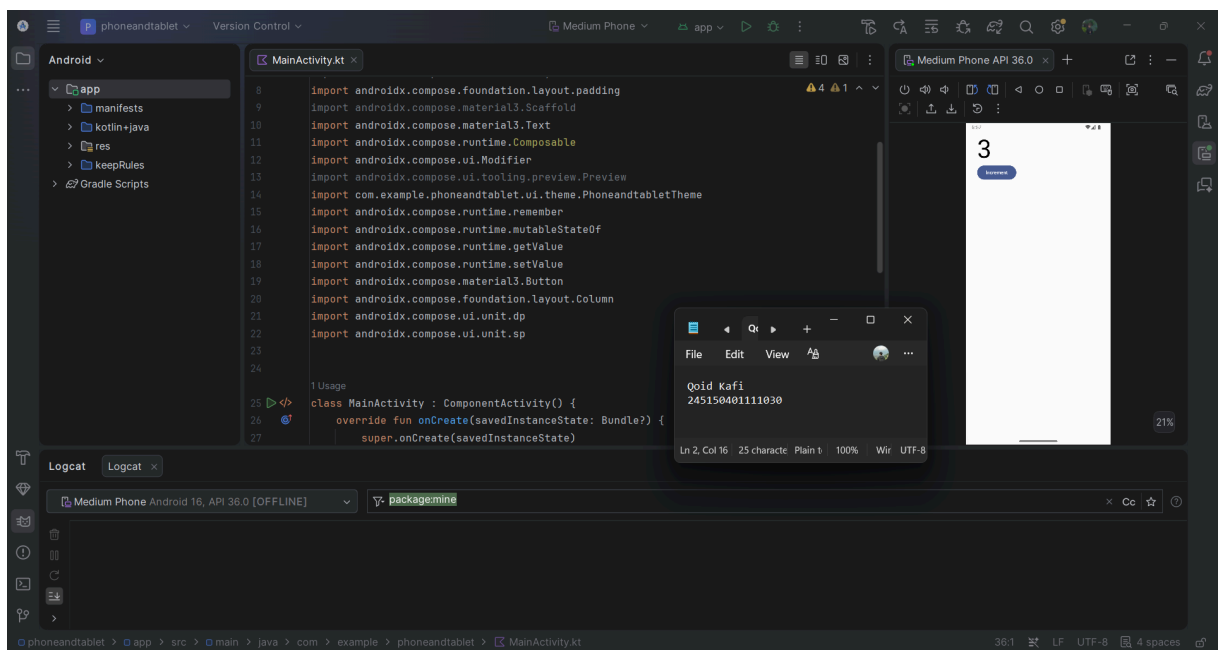
A. State Hoisting

1.

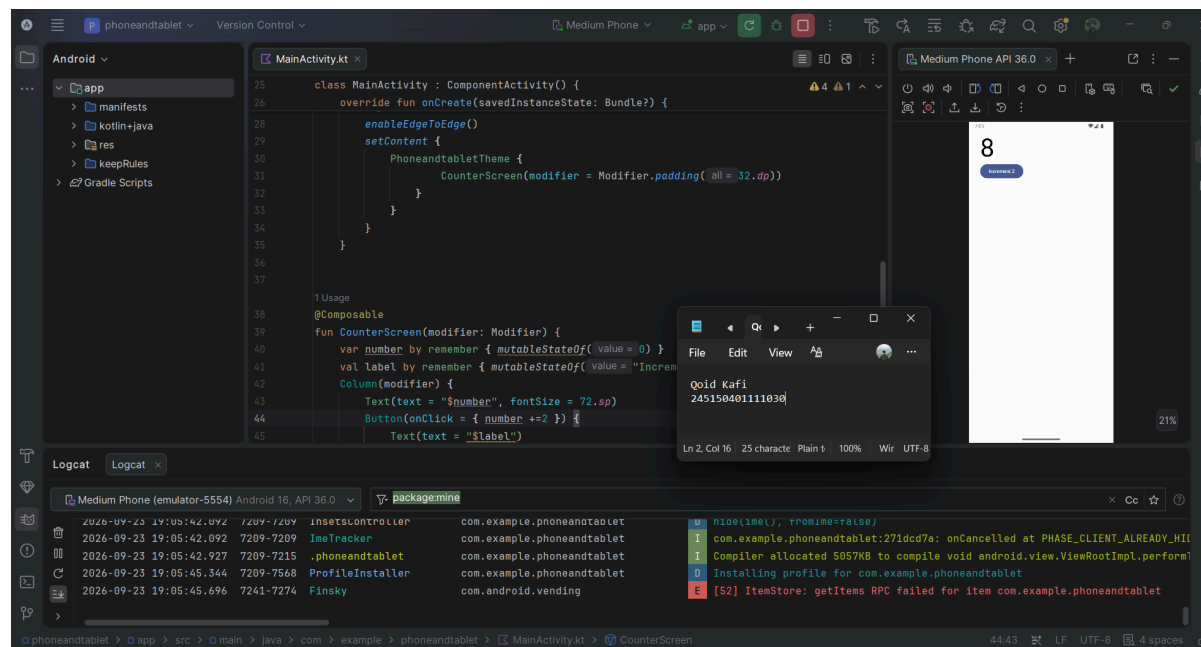
- Sebelum diklik



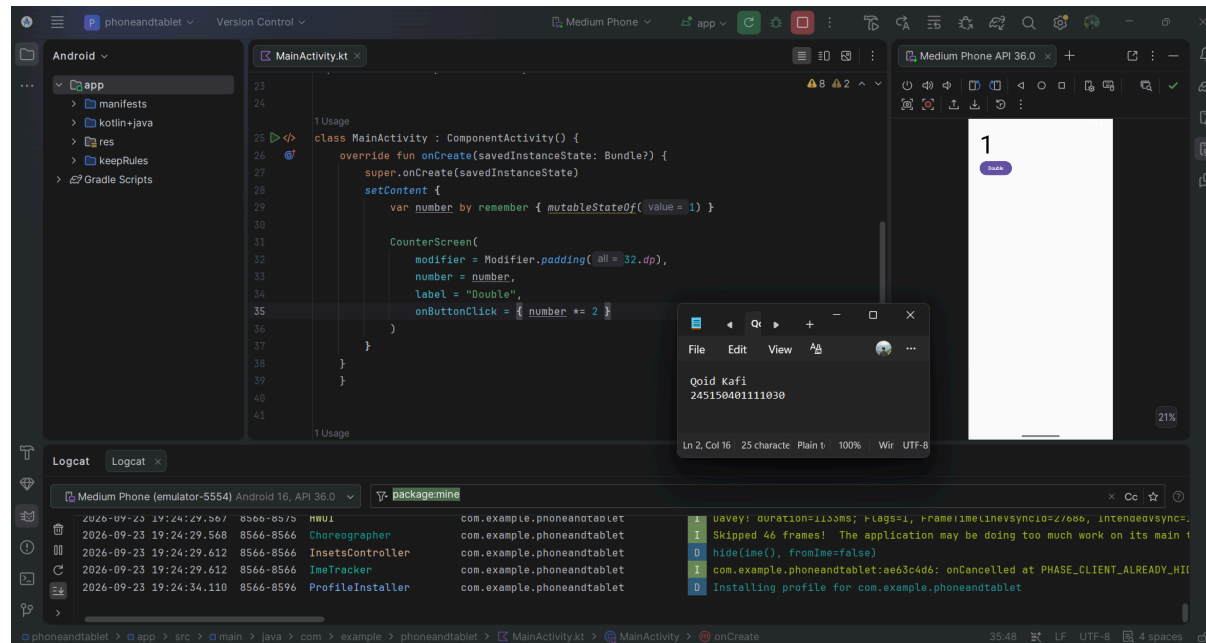
- Sesudah diklik



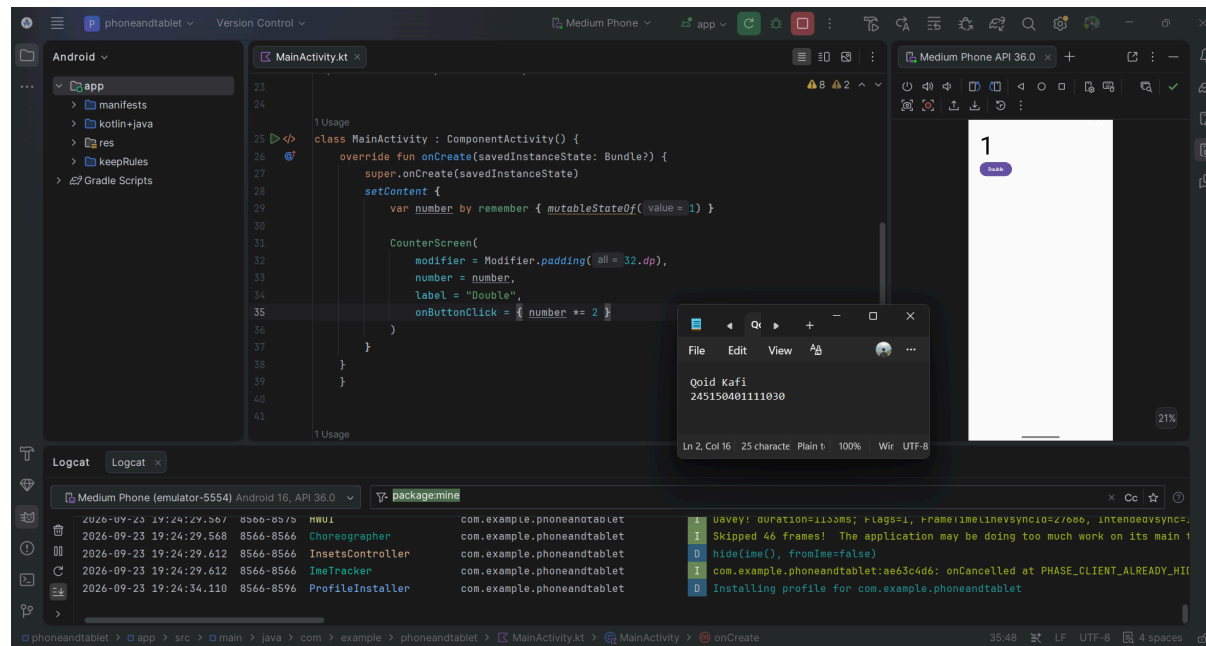
- a. Nilai variabel `number` bertambah 1, dan Jetpack Compose melakukan *recomposition* sehingga angka pada layar langsung diperbarui sesuai nilai terbaru.
 - b. Variabel `number` dideklarasikan menggunakan `mutableStateOf` dan `remember`. Perubahan pada nilai state (`number++`) memicu proses *recomposition* pada Composable `CounterScreen`
 - c. Variabel `number` (`var number by remember { mutableStateOf(0) }`)
 - d. Variabel `label` (`val label by remember { mutableStateOf("Increment") }`).
- 2.
- a. - Mengubah inisialisasi label menjadi "Increment 2".
- Mengubah aksi `onClick` pada Button dari `number++` menjadi `number += 2`
 - b.



- 4.
- a. Ya, aplikasi berjalan secara identik dengan tampilan sebelumnya.
 - b. Di dalam lambda/callback `onButtonClick` pada pemanggilan `CounterScreen` di fungsi `onCreate()`
 - c. Di dalam fungsi `onCreate()` sebelum pemanggilan `CounterScreen`.
6. Alur perubahan pada (XML/View) diawali ketika interaksi pengguna memicu suatu



a.



b.

c. Perubahan logika dan inisialisasi state seluruhnya dilakukan di fungsi onCreate() pada file MainActivity.kt (komponen induk/caller).

d. Tidak ada. Fungsi Composable CounterScreen tidak mengalami perubahan kode sama sekali.

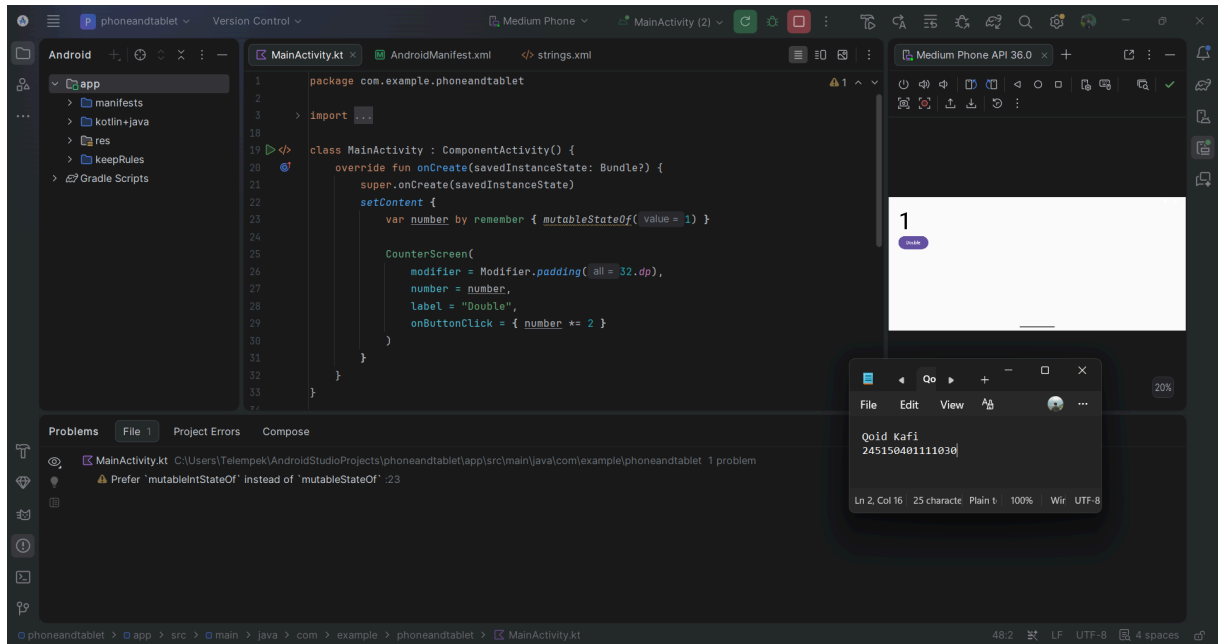
7. Hal ini membuktikan bahwa state hoisting menerapkan prinsip Separation of Concerns (pemisahan logika bisnis dari tampilan). Dengan state hoisting, Composable CounterScreen menjadi stateless dan reusable. Jika terdapat perubahan kebutuhan fungsional atau logika bisnis (seperti mengganti nilai awal, mengubah fungsi dari increment menjadi double), kita cukup mengubah logika di kelas induk (onCreate) tanpa perlu menyentuh atau merusak struktur komponen UI (CounterScreen). Ini

membuat kode lebih mudah dipelihara (maintainable), diuji (testable), dan digunakan kembali (reusable).

B. Perubahan Konfigurasi

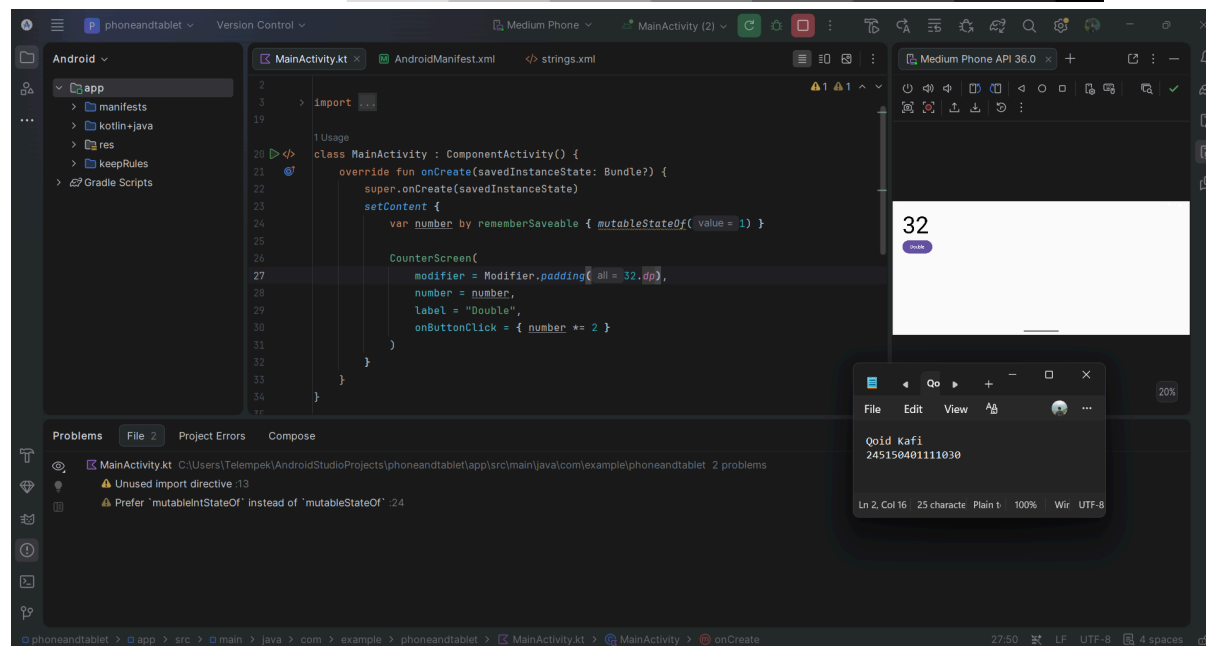
2.

a.



- b. Angka kembali ke 1 (nilai default awal), Saat terjadi perubahan konfigurasi (seperti rotasi layar), sistem Android akan menghancurkan (destroy) Activity lama dan membuatnya ulang (re-create). Karena variabel number menggunakan remember, nilainya hanya disimpan selama recomposition biasa, sehingga nilai state akan hilang dan ter-reset ke nilai awal saat Activity dibuat ulang.





- b. Angka tetap 32 (nilai terakhir sebelum rotasi).
 - c. Karena rememberSaveable menyimpan nilai state ke dalam Bundle (saved instance state). Saat Activity dihancurkan dan dibuat ulang akibat rotasi layar, rememberSaveable secara otomatis memulihkan kembali nilai state yang tersimpan di dalam Bundle tersebut.
4. Contoh Kasus:
- a. Formulir Pendaftaran/Input Teks: Menyimpan data yang diisi pengguna pada TextField (seperti Nama, Email, Alamat).
 - b. Status Checkbox/Radio Button: Menyimpan pilihan filter atau persetujuan syarat & ketentuan.
 - c. Posisi Tab/Scroll List: Menyimpan indeks Tab yang sedang aktif.

C. Launched Effect

1.

- a. Tampilan antarmuka menampilkan angka 1 dan tombol Double, serta pesan Snackbar muncul di bagian bawah layar bertuliskan "Number 1 is shown!"
- b. Karena LaunchedEffect dipanggil saat Composable pertama kali masuk ke dalam komposisi (initial composition). Pada saat ini, nilai variabel number diawali dengan angka 1, sehingga kode di dalam blok LaunchedEffect langsung dipicu.
- c. Pemicunya adalah perubahan atau inisialisasi awal nilai parameter kunci (key) number pada LaunchedEffect(number).

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number
is shown!")
}
```

```
}
```

2.
 - a. Angka pada layar berubah dari 1 menjadi 2, dan SnackBar baru muncul dengan pesan "Number 2 is shown!"
 - b. Ketika tombol diklik, nilai state number berubah (`number *= 2`). Karena number digunakan sebagai key di `LaunchedEffect(number)`, perubahan nilai key ini membatalkan coroutine sebelumnya dan menjalankan ulang blok `LaunchedEffect` dengan nilai number yang baru.
3.
 - a. Tampilan awal menampilkan angka 1 dan SnackBar bertuliskan "Number 1 is shown!"
 - b. etelah 2 detik, angka pada layar berubah dari 1 menjadi 180, dan SnackBar baru otomatis muncul bertuliskan "Number 180 is shown!"
 - c. Pemicunya adalah fungsi `LaunchedEffect(Unit)` yang menjalankan `Repo.getData()` secara asinkron (delay 2 detik) lalu memperbarui nilai number. Perubahan nilai number tersebut kemudian secara otomatis memicu `LaunchedEffect(number)` untuk menampilkan SnackBar baru.

```
LaunchedEffect(Unit) { number = Repo.getData() }
```
4.
 - a.
 - i. `LaunchedEffect(Unit)`: Hanya dijalankan satu kali saja pada saat Composable pertama kali masuk ke dalam komposisi (initial composition). Nilai Unit bersifat konstan sehingga tidak akan pernah memicu eksekusi ulang.
 - ii. `LaunchedEffect(state)`: Dijalankan pada saat initial composition, dan akan dijalankan ulang (re-launch) setiap kali nilai dari parameter state (key) tersebut mengalami perubahan.
 - b.
 - i. `LaunchedEffect(Unit)`: Dijalankan saat pertama kali layar/Composable dibuka (cocok untuk memanggil API/mengambil data awal dari database).
 - ii. `LaunchedEffect(state)`: Dijalankan setiap kali nilai state kunci berubah (cocok untuk efek samping reaktif seperti menampilkan Toast/SnackBar, animasi, atau navigasi berdasarkan perubahan data tertentu).